

Module 14 - chapter 8

EKS cluster information

The project files used in this lecture can be found here:

Terraform:

- <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/terraform/-/tree/feature/eks>

Python:

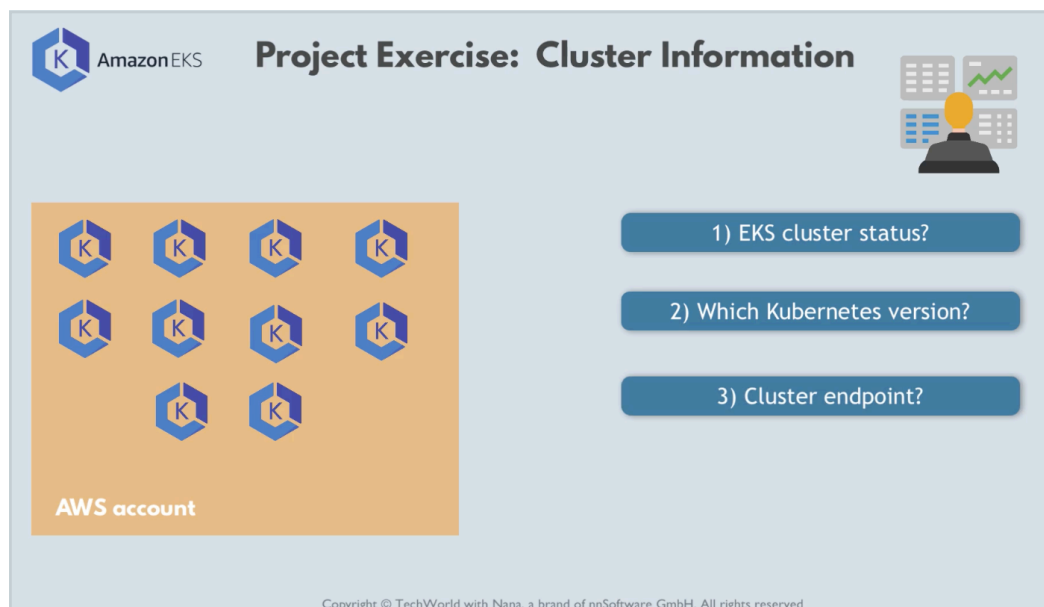
- <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/eks-status-checks.py>

Project intro

Now let's use EKS cluster instead of just working with EC2 servers in isolation.

If we have 10 EKS clusters in our AWS account that were created with TF and we want an overview of the clusters to know:

- EKS cluster status -> like 'Active' or 'Failed', etc
- K8s version each cluster is using
- Cluster endpoint



The slide is titled "Project Exercise: Cluster Information" and features the Amazon EKS logo in the top left. On the left side, there is a large orange rectangle labeled "AWS account" containing ten blue hexagonal icons, each with a white 'K', representing EKS clusters. On the right side, there are three blue buttons with white text, each preceded by a number: "1) EKS cluster status?", "2) Which Kubernetes version?", and "3) Cluster endpoint?". In the top right corner, there is a small icon of a person with a yellow head and a grey body, standing next to a grey grid with a green line graph. At the bottom of the slide, there is a small copyright notice: "Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved".

So we can do this with python

Project Exercise: Cluster Information

1) EKS cluster status?

```

/Users/nana/PycharmProjects/my-python-project/venv/bin/python /Users/nana/
Cluster myapp-eks-cluster status is ACTIVE
Cluster endpoint: https://830450197787f6416189c4696e490003.gc7.eu-central-
Cluster version: 1.28
Process finished with exit code 0
  
```

And also with python we can easily add a web interface to display the information to the users in a dashboard or more friendly way than our terminal

Project Exercise: Cluster Information

1) EKS cluster status?

2) Which Kubernetes version?

3) Cluster endpoint?

UI

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

1. Create EKS cluster (we use https://github.com/AstridCaballero/Module_12_Terraform/blob/Module_12/Terraform_chapter-20/eks-cluster.tf from Module 12 chapter 19 - 21 and I created a new repo here ...)
2. Write Python program

Create EKS cluster with TF

- > terraform init
- > terraform plan

```
terraform > eks-cluster.tf
eks-cluster.tf x vpc.tf x
1 module "eks" {
2   source = "terraform-aws-modules/eks/aws"
3   version = "21.23.0"
4
5   name = "myapp-eks-cluster"
6   kubernetes_version = "1.36" You, A minute ago • Uncommitted changes
7   endpoint_public_access = true
8
9   subnet_ids = module.myapp-vpc.private_subnets
10  vpc_id = module.myapp-vpc.vpc_id
11
12  tags = {
13    environment = "development"
14    application = "myapp"
15  }
16}

module eks > kubernetes_version
Terminal: zsh x + v
# module.eks.module.eks_managed_node_group["dev"].module.user_data.null_resource
+ resource "null_resource" "validate_cluster_service_cidr" {
+   id = (known after apply)
+ }

Plan: 56 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee
apply" now.
```

The above had errors, reason -> it was missing the addons in particular vpc-cni add on:

```

18     addons = {
19         coredns = {}
20         eks-pod-identity-agent = {
21             before_compute = true
22         }
23         kube-proxy = {}
24         vpc-cni = {
25             before_compute = true
26         }
27     }

```

-> terraform apply -auto-approve

Cluster info

Status Active	Kubernetes version 1.36	Support period Standard support until August 2, 2027	Provider EKS
Cluster health 0	Upgrade insights 5	Node health issues 0	Capability issues 0

Nodes (3)

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-10-0-1-236.eu-west-2.compute.internal	t2.small	Node group	dev-7a3e3d780b03e881177ea49e8a	24 minutes ago	Ready	31%	28%	15%
ip-10-0-2-142.eu-west-2.compute.internal	t2.small	Node group	dev-7a3e3d780b03e881177ea49e8a	24 minutes ago	Ready	31%	28%	15%
ip-10-0-3-128.eu-west-2.compute.internal	t2.small	Node group	dev-7a3e3d780b03e881177ea49e8a	24 minutes ago	Ready	21%	24%	15%

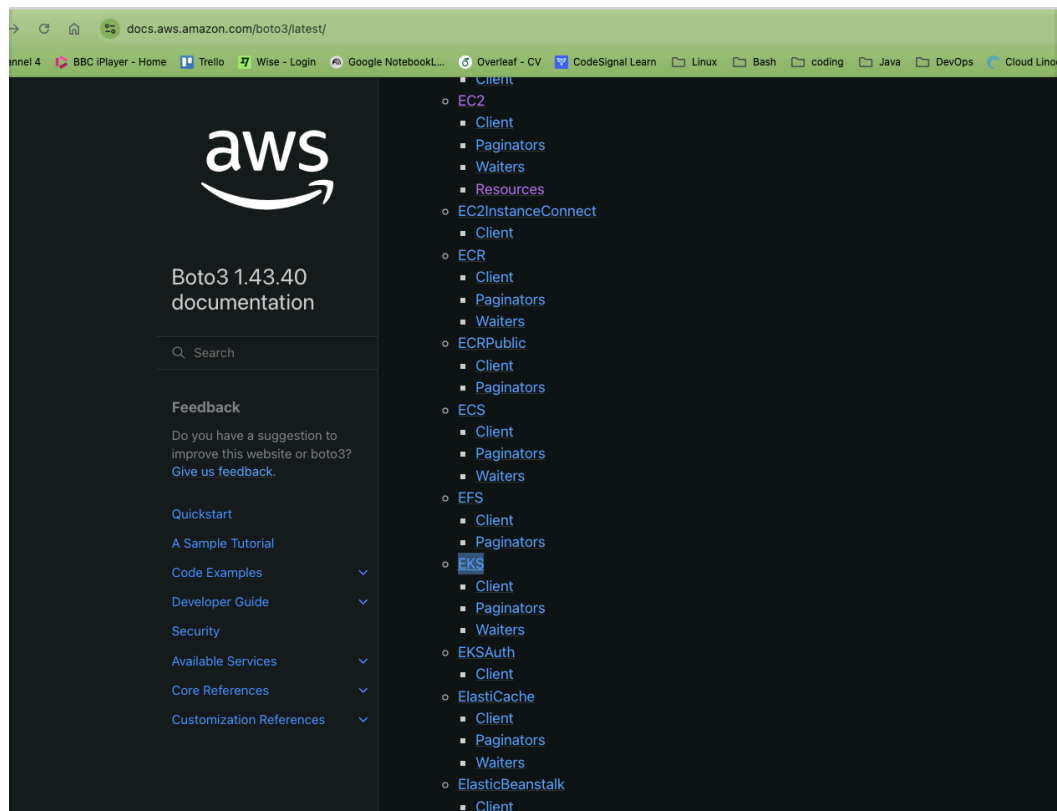
Node groups (1)

Group name	Desired size	AMI release version	Launch template	Status
dev-7a3e3d780b03e881177ea49e8a	3	1.36.2-20260625	dev-d32819504ce7b5038e0c3e3964 (1)	Active

Now we can write our python program for cluster health checks

Implementation

And we use the EKS client <https://docs.aws.amazon.com/boto3/latest/>



EKS

Client

class `EKS.Client`

A low-level client representing Amazon Elastic Kubernetes Service (EKS)

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed service that makes it easy for you to run Kubernetes on Amazon Web Services without needing to setup or maintain your own Kubernetes control plane. Kubernetes is an open-source system for automating the deployment, scaling, and management of containerized applications.

Amazon EKS runs up-to-date versions of the open-source Kubernetes software, so you can use all the existing plugins and tooling from the Kubernetes community. Applications running on Amazon EKS are fully compatible with applications running on any standard Kubernetes environment, whether running in on-premises data centers or public clouds. This means that you can easily migrate any standard Kubernetes application to Amazon EKS without any code modification required.

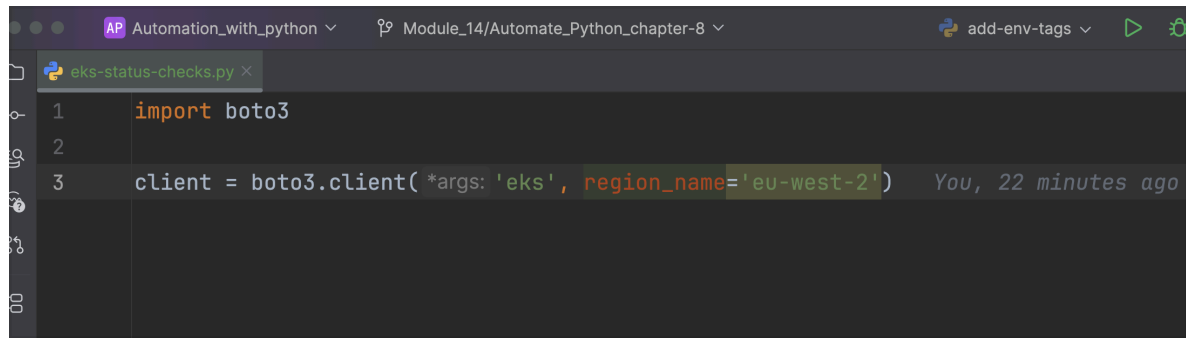
```
import boto3

client = boto3.client('eks')
```

These are the available methods:

- `associate_access_policy`
- `associate_encryption_config`
- `associate_identity_provider_config`

And here is the code in my program



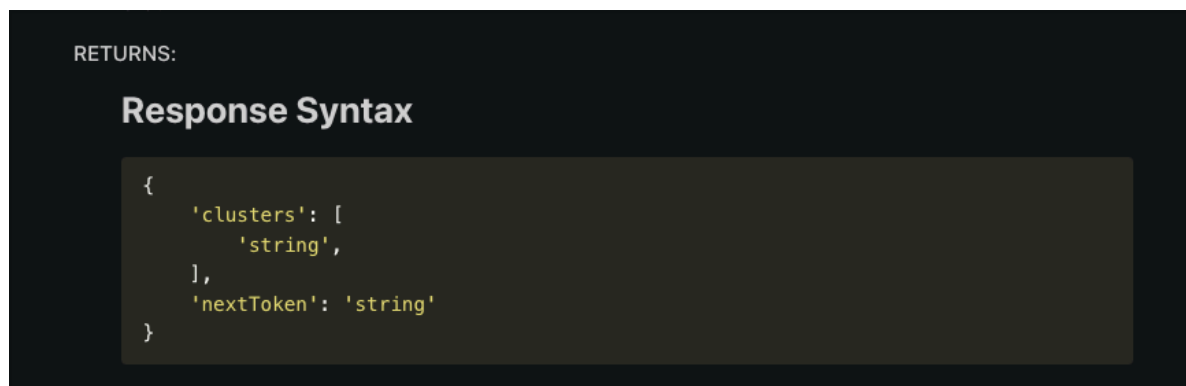
```
1 import boto3
2
3 client = boto3.client(*args: 'eks', region_name='eu-west-2')
```

The screenshot shows a code editor with a dark theme. The top bar indicates the project is 'Automation_with_python' and the file is 'Module_14/Automate_Python_chapter-8'. The active file is 'eks-status-checks.py'. The code consists of three lines: importing boto3, and creating a boto3 client for the 'eks' service in the 'eu-west-2' region. A timestamp 'You, 22 minutes ago' is visible next to the third line.

We want to collect all the clusters then iterate over each and check their status

So we will collect the clusters in a list with function -> `list_clusters()`
https://docs.aws.amazon.com/boto3/latest/reference/services/eks/client/list_clusters.html

This is the response



```
RETURNS:

Response Syntax

{
  'clusters': [
    'string',
  ],
  'nextToken': 'string'
}
```

The screenshot shows a dark-themed box with the text 'RETURNS:' followed by a section titled 'Response Syntax'. Below the title is a JSON-like structure representing the response format: a dictionary with a 'clusters' key (a list of strings) and a 'nextToken' key (a string).

And here is the code

```
eks-status-checks.py x
1 import boto3
2
3 client = boto3.client(*args: 'eks', region_name='eu-west-2')
4 clusters = client.list_clusters()
5 print(clusters) You, 3 minutes ago • Uncommitted changes

Run eks-status-checks x
/Users/astri...
{'ResponseMetadata': {'RequestId': 'bd7e56ae-20a0-43e2-924c-3900ce78e9f5', 'HTTP...
Process finished with exit code 0
```

We want only the 'clusters' value only we don't want the 'nextToken' value or any other data -> clusters['clusters']

```
eks-status-checks.py x
1 import boto3
2
3 client = boto3.client(*args: 'eks', region_name='eu-west-2')
4 clusters = client.list_clusters()
5 print(clusters['clusters']) You, 3 minutes ago • Uncommitted changes

Run eks-status-checks x
/Users/astri...
['myapp-eks-cluster']
Process finished with exit code 0
```

We got a list with one cluster because we only have one cluster in eu-west-2. And it has the name that we defined in TF

We can now use the name of the cluster to fetch info from each cluster.

And we need a function to get the cluster information and EKS service has a function called describe_cluster

docs.aws.amazon.com/boto3/latest/reference/services/eks.html

BBC iPlayer - Home

Trello

Wise - Login

Google NotebookL...

Overleaf - CV

CodeSignal Learn

Linux

Bash

coding

aws

Boto3 1.43.42

documentation

Q Search

Feedback

Do you have a suggestion to improve this website or boto3? Give us feedback.

Quickstart

A Sample Tutorial

Code Examples

Developer Guide

Security

Available Services

AccessAnalyzer

Account

ACM

ACMPCA

AIOps

• [associate_encryption_config](#)

• [associate_identity_provider_config](#)

• [can_paginate](#)

• [cancel_update](#)

• [close](#)

• [create_access_entry](#)

• [create_addon](#)

• [create_capability](#)

• [create_cluster](#)

• [create_eks_anywhere_subscription](#)

• [create_fargate_profile](#)

• [create_nodegroup](#)

• [create_pod_identity_association](#)

• [delete_access_entry](#)

• [delete_addon](#)

• [delete_capability](#)

• [delete_cluster](#)

• [delete_eks_anywhere_subscription](#)

• [delete_fargate_profile](#)

• [delete_nodegroup](#)

• [delete_pod_identity_association](#)

• [deregister_cluster](#)

• [describe_access_entry](#)

• [describe_addon](#)

• [describe_addon_configuration](#)

• [describe_addon_versions](#)

• [describe_capability](#)

• [describe_cluster](#)

• [describe_cluster_versions](#)

• [describe_eks_anywhere_subscription](#)

• [describe_fargate_profile](#)

• [describe_identity_provider_config](#)

• [describe_insight](#)

• [describe_insights_refresh](#)

• [describe_nodegroup](#)

↑ Back to top

EKS / Client / describe_cluster

describe_cluster

`EKS.Client.describe_cluster(**kwargs)`

Describes an Amazon EKS cluster.

The API server endpoint and certificate authority data returned by this operation are required for `kubelet` and `kubectl` to communicate with your Kubernetes API server. For more information, see [Creating or updating a kubeconfig file for an Amazon EKS cluster](#).

Note

The API server endpoint and certificate authority data aren't available until the cluster reaches the `ACTIVE` state.

See also: [AWS API Documentation](#)

Request Syntax

```
response = client.describe_cluster(
    name='string'
)
```

PARAMETERS:

name (*string*) –

[REQUIRED]

The name of your cluster.

RETURN TYPE:

dict

And requires the name of the cluster, which we already have collected in a list.

```
4 ✦ clusters = client.list_clusters()['clusters'] You, Momen
5
6 for cluster in clusters:
7     response = client.describe_cluster(
8         name=cluster
9     )
```

Now let's see what the function returns:

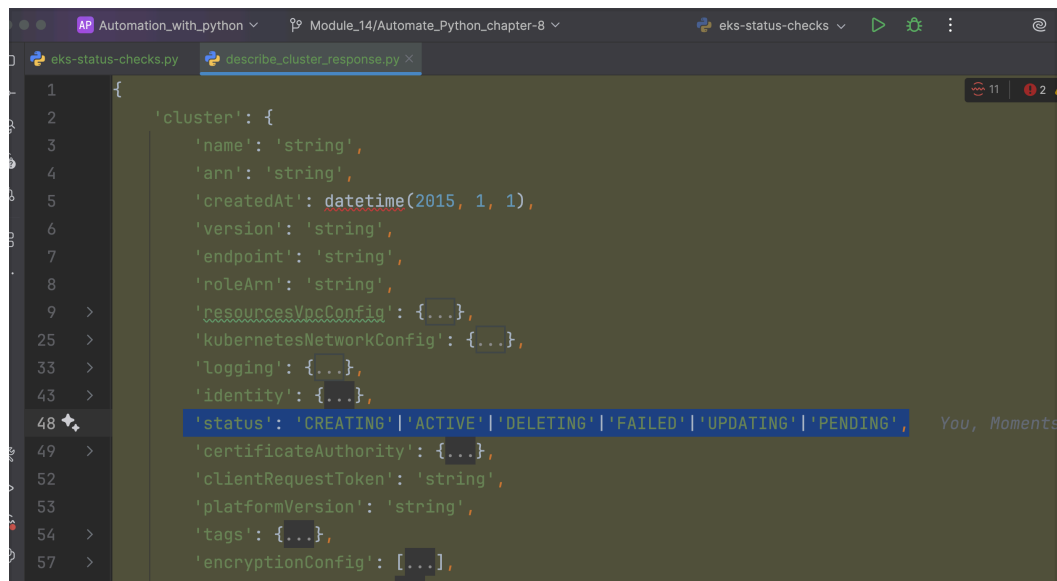
From docs website

RETURNS:

Response Syntax

```
{
  'cluster': {
    'name': 'string',
    'arn': 'string',
    'createdAt': datetime(2015, 1, 1),
    'version': 'string',
    'endpoint': 'string',
    'roleArn': 'string',
    'resourcesVpcConfig': {
      'subnetIds': [
        'string',
      ],
      'securityGroupIds': [
        'string',
      ],
      'clusterSecurityGroupId': 'string',
      'vpcId': 'string',
      'endpointPublicAccess': True|False,
      'endpointPrivateAccess': True|False,
      'publicAccessCidrs': [
        'string',
      ],
    },
  },
}
```

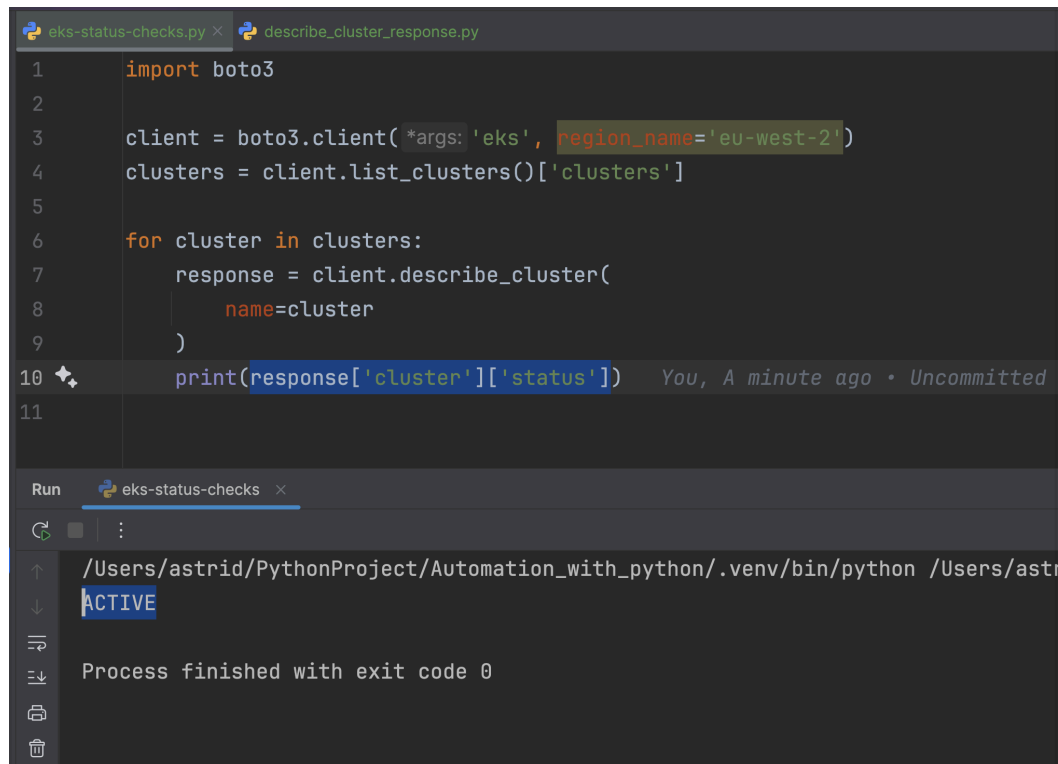
I copy and paste the response into a python file for readability:



```
1 {
2     'cluster': {
3         'name': 'string',
4         'arn': 'string',
5         'createdAt': datetime(2015, 1, 1),
6         'version': 'string',
7         'endpoint': 'string',
8         'roleArn': 'string',
9         'resourcesVpcConfig': {...},
10        'kubernetesNetworkConfig': {...},
11        'logging': {...},
12        'identity': {...},
13        'status': 'CREATING'|'ACTIVE'|'DELETING'|'FAILED'|'UPDATING'|'PENDING',
14        'certificateAuthority': {...},
15        'clientRequestToken': 'string',
16        'platformVersion': 'string',
17        'tags': {...},
18        'encryptionConfig': [...],
19    },
20 }
```

So we get a dictionary and one of the keys is 'cluster' which value is another dictionary and has a key called 'status' in line 48 above 🙏

-> response['cluster']['status']



```
1 import boto3
2
3 client = boto3.client(*args: 'eks', region_name='eu-west-2')
4 clusters = client.list_clusters()['clusters']
5
6 for cluster in clusters:
7     response = client.describe_cluster(
8         name=cluster
9     )
10    print(response['cluster']['status']) You, A minute ago • Uncommitted
11
```

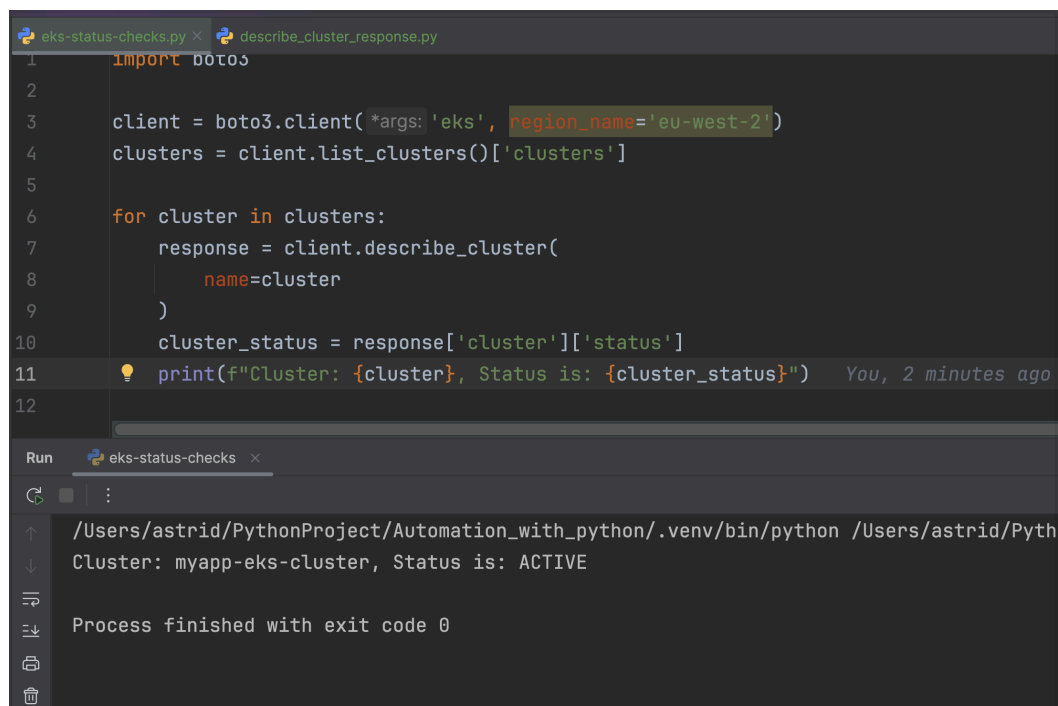
Run eks-status-checks x

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_with_python/eks-status-checks.py

ACTIVE

Process finished with exit code 0

And we refactor it



```
1 import boto3
2
3 client = boto3.client(*args: 'eks', region_name='eu-west-2')
4 clusters = client.list_clusters()['clusters']
5
6 for cluster in clusters:
7     response = client.describe_cluster(
8         name=cluster
9     )
10    cluster_status = response['cluster']['status']
11    print(f"Cluster: {cluster}, Status is: {cluster_status}") You, 2 minutes ago
12
```

Run eks-status-checks x

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_with_python/eks-status-checks.py

Cluster: myapp-eks-cluster, Status is: ACTIVE

Process finished with exit code 0

We also want to have an overview of the cluster endpoint, so let's see if we have it in the describe_cluster() response

```
eks-status-checks.py describe_cluster_response.py x
1 {
2     'cluster': {
3         'name': 'string',
4         'arn': 'string',
5         'createdAt': datetime(2015, 1, 1),
6         'version': 'string',
7         'endpoint': 'string',
8         'roleArn': 'string',
```

And the key is there in line 7 🙏

```
eks-status-checks.py describe_cluster_response.py
3 client = boto3.client(*args='eks', region_name='eu-west-2')
4 clusters = client.list_clusters()['clusters']
5
6 for cluster in clusters:
7     response = client.describe_cluster(
8         name=cluster
9     )
10    cluster_info = response['cluster']
11    cluster_status = cluster_info['status']
12    cluster_endpoint = cluster_info['endpoint']
13
14    print(f"Cluster {cluster}, Status is {cluster_status}")
15    print(f"Cluster endpoint: {cluster_endpoint}")
16
17

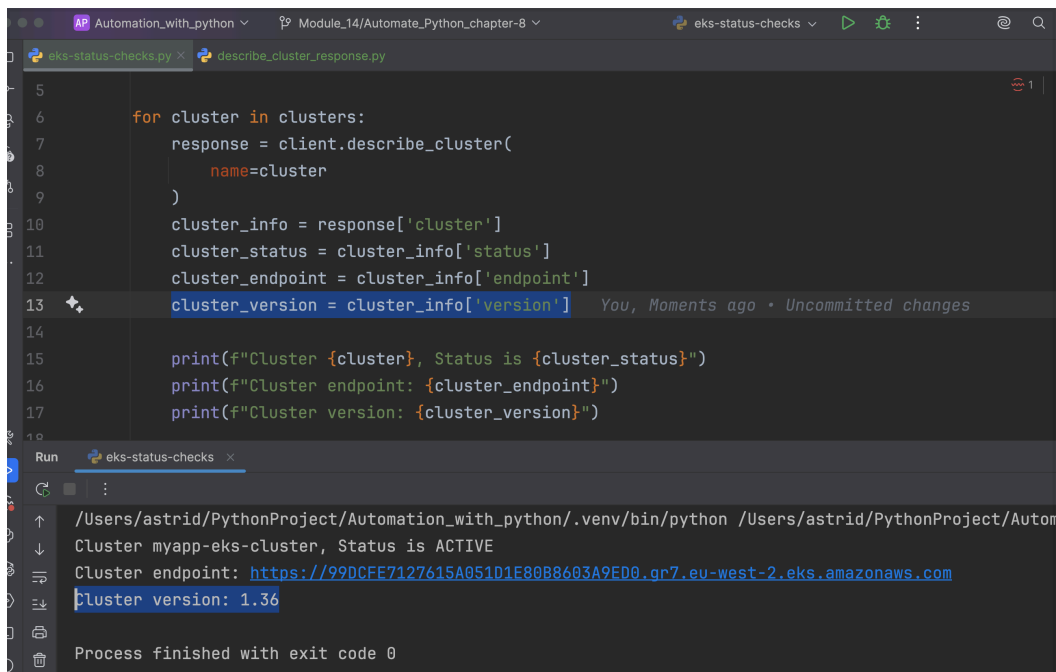
Run eks-status-checks x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/A
Cluster myapp-eks-cluster, Status is ACTIVE
Cluster endpoint: https://99DCFE7127615A051D1E80B8603A9ED0.gr7.eu-west-2.eks.amazonaws.com
Process finished with exit code 0
```

And the last piece of info we want is the Kubernetes version.



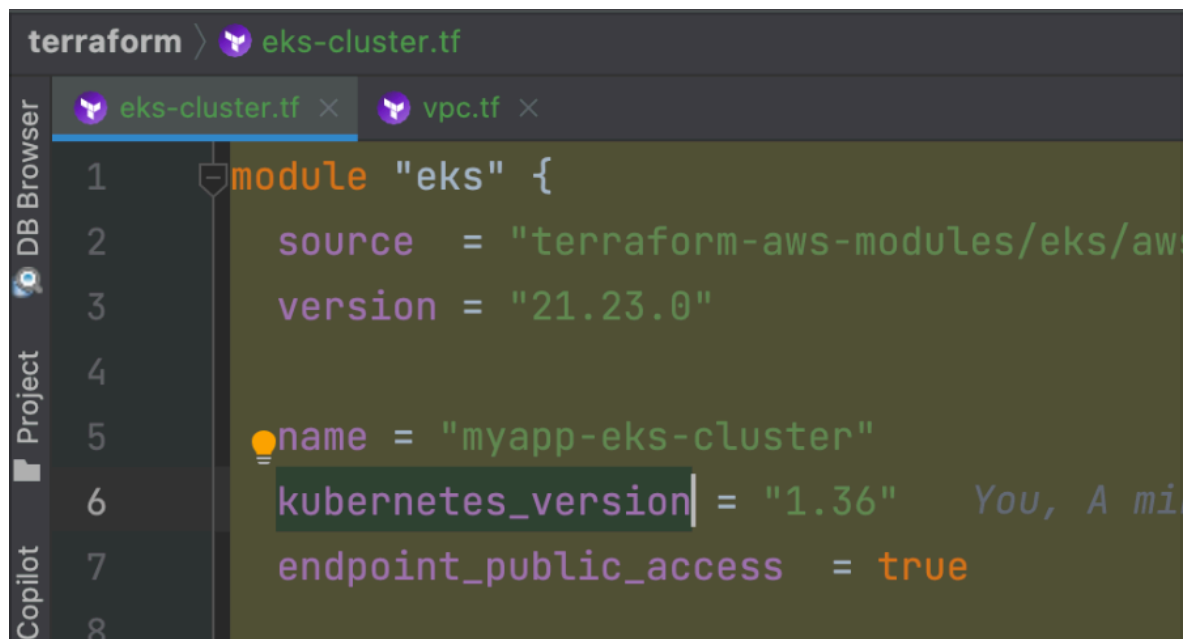
```
eks-status-checks.py describe_cluster_response.py x
1 {
2     'cluster': {
3         'name': 'string',
4         'arn': 'string',
5         'createdAt': datetime(2015, 1, 1),
6         'version': 'string',
7         'endpoint': 'string',
```

So the code is now:



```
Automation_with_python Module_14/Automate_Python_chapter-8 eks-status-checks
eks-status-checks.py describe_cluster_response.py
5
6 for cluster in clusters:
7     response = client.describe_cluster(
8         name=cluster
9     )
10    cluster_info = response['cluster']
11    cluster_status = cluster_info['status']
12    cluster_endpoint = cluster_info['endpoint']
13    cluster_version = cluster_info['version']
14
15    print(f"Cluster {cluster}, Status is {cluster_status}")
16    print(f"Cluster endpoint: {cluster_endpoint}")
17    print(f"Cluster version: {cluster_version}")
18
Run eks-status-checks
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Autom
Cluster myapp-eks-cluster, Status is ACTIVE
Cluster endpoint: https://99DCFE7127615A051D1E80B8603A9F00.gr7.eu-west-2.eks.amazonaws.com
Cluster version: 1.36
Process finished with exit code 0
```

Which is the version defined in our TF see line 6 📌

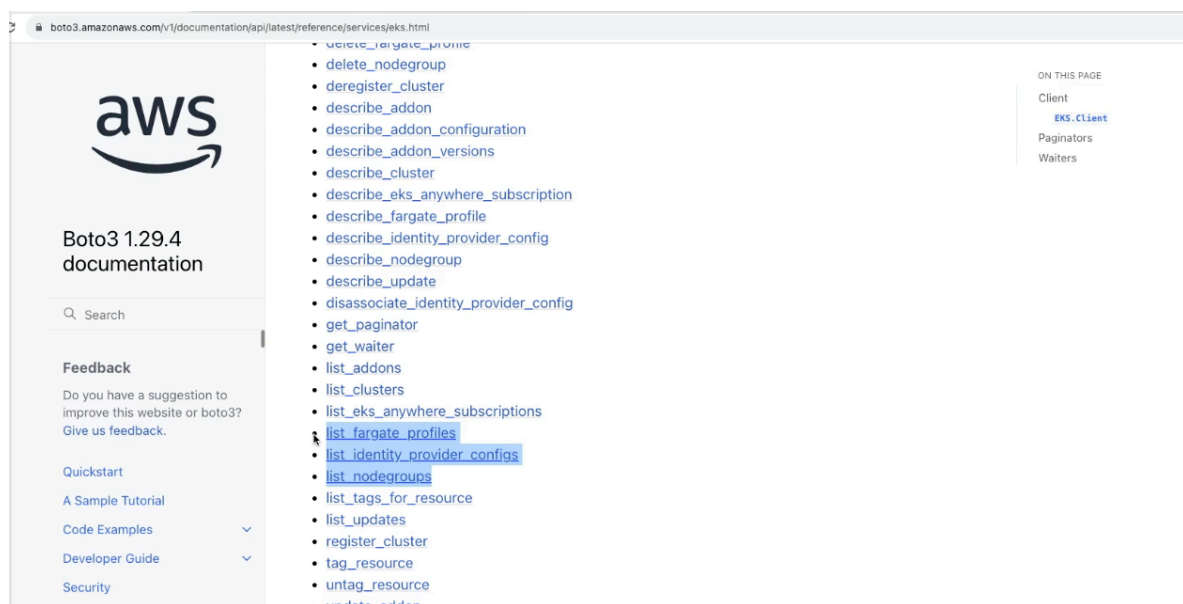


The screenshot shows a code editor with two tabs: 'eks-cluster.tf' (active) and 'vpc.tf'. The 'eks-cluster.tf' tab contains the following Terraform code:

```
1 module "eks" {
2     source = "terraform-aws-modules/eks/aws"
3     version = "21.23.0"
4
5     name = "myapp-eks-cluster"
6     kubernetes_version = "1.36"
7     endpoint_public_access = true
8 }
```

The left sidebar of the editor shows icons for 'DB Browser', 'Project', and 'Copilot'.

If our EKS uses fargate and we want that information or the nodegroup information we can find the functions to get the names of the fargate profiles or the nodegroups from the EKS service as well:



And then we can also use `describe_fargate_profile` and `describe_nodegroup` to get the information using the names like we did with `list_clusters()` and `describe_cluster()`